FACULTY OF ENGINEERING SCIENCES AND TECHNOLOGY

HAMDARD UNIVERSITY KARACHI

Department of Information Technology (DIT)

Laboratory Manual

Infrastructure Automation (CCIA-5214)

Course Level	Undergraduate – Semester 6
Credit Hours	2 + 1 (Theory + Lab)
Term	Fall 2026
Degree Program	Cloud Application Development and Operations
Instructor	Iqbal Uddin Khan
Email	iqbaluddin.khan@hamdard.edu.pk
Total Labs	12 (aligned to the course outline's Lab Task – CLO Mapping table)
Tooling Policy	All software used in this manual is free and open source; no paid cloud accounts are required.

Table of Contents

Table of Contents **2**
Introduction to This Manual **3**
How to Use This Manual 3
Host Ports Used by the Labs 3
Lab 1: Introduction to Infrastructure as Code (IaC) **5**
1. Objectives 5
2. Preamble / Short Theory 5
3. Prerequisites 5
4. Step-by-Step Guide 5
5. Required Outcomes / Tasks from the Student 7
Lab 2: Writing Basic Automation Scripts **8**
1. Objectives 8
2. Preamble / Short Theory 8
3. Prerequisites 8
4. Step-by-Step Guide 8
5. Required Outcomes / Tasks from the Student 9
Lab 3: Configuration Management with Ansible **11**
1. Objectives 11
2. Preamble / Short Theory 11
3. Prerequisites 11
4. Step-by-Step Guide 11
5. Required Outcomes / Tasks from the Student 14
Lab 4: Automating Deployments with Puppet (OpenVox) **15**
1. Objectives 15
2. Preamble / Short Theory 15
3. Prerequisites 15
4. Step-by-Step Guide 15
5. Required Outcomes / Tasks from the Student 16
Lab 5: Container Orchestration Basics **18**
1. Objectives 18
2. Preamble / Short Theory 18
3. Prerequisites 18
4. Step-by-Step Guide 18
5. Required Outcomes / Tasks from the Student 19
Lab 6: Infrastructure Provisioning with Terraform / OpenTofu **20**
1. Objectives 20
2. Preamble / Short Theory 20
3. Prerequisites 20
4. Step-by-Step Guide 20
5. Required Outcomes / Tasks from the Student 22
Lab 7: CI/CD Pipeline Automation **23**
1. Objectives 23
2. Preamble / Short Theory 23
3. Prerequisites 23
4. Step-by-Step Guide 23

5. Required Outcomes / Tasks from the Student....25
Lab 8: Automated Monitoring and Logging....27
1. Objectives....27
2. Preamble / Short Theory....27
3. Prerequisites....27
4. Step-by-Step Guide....27
5. Required Outcomes / Tasks from the Student....28
Lab 9: Security in Infrastructure Automation....30
1. Objectives....30
2. Preamble / Short Theory....30
3. Prerequisites....30
4. Step-by-Step Guide....30
5. Required Outcomes / Tasks from the Student....31
Lab 10: Multi-Cloud Automation Case Study....32
1. Objectives....32
2. Preamble / Short Theory....32
3. Prerequisites....32
4. Step-by-Step Guide....32
5. Required Outcomes / Tasks from the Student....35
Lab 11: Disaster Recovery Automation....36
1. Objectives....36
2. Preamble / Short Theory....36
3. Prerequisites....36
4. Step-by-Step Guide....36
5. Required Outcomes / Tasks from the Student....38
Lab 12: Capstone Project: Design and Implement an IaC Solution....39
1. Objectives....39
2. Preamble / Short Theory....39
3. Prerequisites....39
4. Step-by-Step Guide....39
5. Required Outcomes / Tasks from the Student....40

Introduction to This Manual

This laboratory manual accompanies the CCIA-5214 Infrastructure Automation course and provides twelve hands-on labs aligned to the course outline's Lab Task – CLO Mapping table. Each lab follows the same structure: Objectives, Preamble / Short Theory, Prerequisites, a Step-by-Step Guide, and the Required Outcomes a student must submit.

Open-source tooling policy. Every tool used in this manual is free and open source. Where the course outline names a tool that is proprietary or no longer distributed under an open-source license, an open-source equivalent is used, as summarized below.

Item in course outline	Approach used in this manual
Terraform (Labs 1, 6, 10, 12)	OpenTofu (MPL-2.0), command `tofu`. Terraform releases from 1.6 onward use the Business Source License, which is source-available rather than open source. Syntax and workflow are identical; replace `tofu` with `terraform` if you prefer.
Puppet Open Source (Lab 4)	OpenVox, the community-maintained open-source distribution. The `puppet` command and manifest language are unchanged.
GitHub Actions (Lab 7)	Jenkins (self-hosted, open source) is the primary CI/CD tool; the GitHub Actions free tier is an optional alternative.
Velero with Restic (Lab 11)	Velero's file-system backup now uses Kopia (Restic was deprecated and removed from its backup path). Restic is practiced as a standalone tool.
AWS CloudFormation (Week 7) and Google Cloud Deployment Manager (Week 9)	Theory only. No lab requires a proprietary cloud service; auto-scaling is practiced with Kubernetes.

How to Use This Manual

- Complete the labs in sequence. Later labs (6, 7, 8, 10, 11, 12) build on the OpenTofu, Docker, and Minikube environments from Labs 1, 3, and 5.
- Commands are written for Linux and macOS. Windows users should run Bash, Docker, and OpenTofu commands inside WSL2 (free) unless noted.
- Replace placeholders such as `Your Name - Roll No` and `<one-of-your-pod-names>` with your own values.
- When a step refers to a previous lab's files or cluster, make sure they still exist or re-create them first.
- Every lab ends with an AI-restricted submission policy; read it before you start so you capture the required evidence as you work.

Host Ports Used by the Labs

To avoid clashes, each lab uses its own ports. If a port is already in use, stop the earlier lab's containers first.

Lab	Host ports	Purpose
3	2221, 2222, 8181, 8182	SSH and HTTP for the two Ansible target containers
6	8080 and up	Web containers (one port per container)
7	8090, 50000	Jenkins web interface and agent port
8	9090, 9100, 3000	Prometheus, node-exporter, Grafana
11	9000, 9001	MinIO API and console
12	8088, 9091, 8090	Load balancer, Prometheus, Jenkins

Lab 1: Introduction to Infrastructure as Code (IaC)

Week	1
Mapped CLO	CLO-1 (Understand)
Tools / Environment	OpenTofu (open source) or Terraform; Git; OS terminal
Estimated Time	90 minutes

1. Objectives

On successful completion of this lab, the student will be able to:

- Explain the principles of Infrastructure as Code and how it differs from manual provisioning.
- Install and configure OpenTofu and Git.
- Initialize a version-controlled project, apply a first minimal configuration, and demonstrate idempotency and drift detection.

2. Preamble / Short Theory

Infrastructure as Code (IaC) treats server, network, and platform configuration as versioned source files rather than manual console clicks, making infrastructure repeatable, auditable, and reviewable like application code. This course uses OpenTofu, the MPL-2.0 open-source Terraform-compatible tool, paired with Git for version control. The commands are identical to Terraform's; if you already have Terraform installed, replace `tofu` with `terraform`.

This first lab establishes the workflow that every later lab builds on: write, initialize, plan, apply, and destroy. You will also see the two properties that make IaC safe: a second apply with no changes does nothing (idempotency), and a manual edit to a managed resource is detected as drift.

3. Prerequisites

- A laptop or desktop running Windows, macOS, or Linux with permission to install software (Windows users: use WSL2).
- No prior automation experience is assumed.
- A stable internet connection for the initial downloads.

4. Step-by-Step Guide

Step 1: Install OpenTofu

Follow the instructions at opentofu.org/docs/intro/install, or use a package manager:

```
# macOS
brew install opentofu

# Ubuntu / WSL2
sudo snap install --classic opentofu

# Windows (PowerShell)
choco install opentofu
```

Step 2: Install Git and set your identity

Install Git from git-scm.com or your package manager, then tell Git who you are (commits fail without this).

```
sudo apt install -y git        # or: brew install git / winget install
Git.Git
git config --global user.name  "Your Name"
git config --global user.email "you@example.com"
```

Step 3: Verify both installations

```
tofu version
git --version
```

Step 4: Initialize a Git repository

```
mkdir lab01-iac-intro && cd lab01-iac-intro
git init -b main
printf '.terraform/\n*.tfstate\n*.tfstate.*\n' > .gitignore
```

Step 5: Write a minimal configuration

Create `main.tf`. It uses the local provider, so no cloud account is needed. Replace the default value of `student_name` with your own name and roll number.

```
terraform {
  required_version = ">= 1.6.0"
  required_providers {
    local = {
      source  = "hashicorp/local"
      version = "~> 2.4"
    }
  }
}

variable "student_name" {
  description = "Your full name and roll number"
  type        = string
  default     = "Your Name (Roll No)"
}

resource "local_file" "hello" {
  filename = "${path.module}/hello.txt"
  content  = "Provisioned by OpenTofu for ${var.student_name}\n"
}
```

Step 6: Initialize, plan, and apply

```
tofu init
tofu plan
tofu apply
cat hello.txt
```

Read the plan before typing `yes`. Note the symbols + (create), ~ (change), and - (destroy).

Step 7: Prove idempotency

Run `tofu apply` a second time without changing anything and record the exact message that OpenTofu prints.

Step 8: Detect and correct drift

Edit `hello.txt` by hand, then run `tofu plan`. Observe how OpenTofu detects that reality no longer matches the code, and run `tofu apply` to restore it.

```
echo 'tampered' > hello.txt
tofu plan
tofu apply
```

Step 9: Inspect the state file

List and inspect what OpenTofu tracks, then open `terraform.tfstate` (OpenTofu keeps this historic file name). Explain in your own words why the state file must not be edited by hand.

```
tofu state list
tofu state show local_file.hello
```

Step 10: Commit the configuration

```
git add main.tf .gitignore .terraform.lock.hcl
git commit -m "Lab 1: first OpenTofu configuration"
git log --oneline
```

Step 11: Destroy the resource

```
tofu destroy
```

5. Required Outcomes / Tasks from the Student

- `main.tf` and a screenshot of `git log --oneline` showing your commit.
- A dated terminal screenshot showing a full init, plan, apply, drift-fix, and destroy cycle.
- The exact text printed by the second `apply` (Step 7) and your one-paragraph explanation of what the state file is for.

> **AI-restricted submission policy**
> All deliverables must include a dated screenshot (system clock visible) of your own terminal or IDE output, with your name and roll number visible in the prompt, filename, or a code comment. Where indicated, a short live demonstration or live search may be required in the lab session. Work that cannot be reproduced live by the student will not be marked.

Lab 2: Writing Basic Automation Scripts

Week	3
Mapped CLO	CLO-2 (Apply)
Tools / Environment	Python 3; Bash (GNU); VS Code or VSCodium
Estimated Time	90 minutes

1. Objectives

On successful completion of this lab, the student will be able to:

- Write basic Python and Bash scripts that automate a repetitive infrastructure task.
- Compare scripting idioms between Python and Bash for the same task.
- Parameterize scripts with command-line arguments and environment variables.

2. Preamble / Short Theory

Before adopting dedicated configuration-management or IaC tools, engineers often automate small tasks with plain scripts: checking disk space, rotating logs, creating directory structures. This lab builds that scripting foundation using only free, built-in language features and reinforces the automation maturity model from Week 1 (ad-hoc scripts to standardized scripts).

3. Prerequisites

- Lab 1 completed.
- Python 3 and a Bash-capable shell (native on Linux and macOS; WSL2 on Windows).

4. Step-by-Step Guide

Step 1: Write a Bash disk-usage check

Save as `disk_check.sh` and make it executable with `chmod +x disk_check.sh`.

```
#!/usr/bin/env bash
THRESHOLD=80
USAGE=$(df / | tail -1 | awk '{print $5}' | tr -d '%')
if [ "$USAGE" -ge "$THRESHOLD" ]; then
  echo "WARNING: disk usage at ${USAGE}%"
else
  echo "OK: disk usage at ${USAGE}%"
fi
```

Step 2: Write the equivalent in Python

Save as `disk_check.py`.

```
import shutil

THRESHOLD = 80
total, used, free = shutil.disk_usage("/")
percent = used / total * 100
print(f"{'WARNING' if percent >= THRESHOLD else 'OK'}: disk usage at
{percent:.1f}%")
```

Run both scripts and compare their output. They differ slightly because `df` and `shutil.disk_usage` calculate the used percentage differently; note the difference and explain it in one sentence.

Step 3: Parameterize both scripts

Accept the threshold as a command-line argument in each version instead of the hard-coded constant, and make the scripts exit with a non-zero status when the threshold is exceeded so that other tools can react to the result.

Step 4: Use environment variables

Extend both scripts so that, when no argument is supplied, the threshold is read from the environment variable `DISK_THRESHOLD`, falling back to 80 if it is not set.

```
DISK_THRESHOLD=50 ./disk_check.sh
./disk_check.sh 10
./disk_check.sh 95
```

Step 5: Automate bulk directory creation

Create `hosts.txt` and write a script (either language) that reads it and creates `<host>/logs` and `<host>/configs` for every host. Blank lines and lines beginning with # must be ignored, and running the script twice must not fail (idempotency).

```
web01
web02
db01
# comments and blank lines are ignored

cache01
```

Step 6: Compare the two languages

Write a short comparison (bullet points are fine) of readability, portability, and error handling between your Bash and Python versions.

5. Required Outcomes / Tasks from the Student

- Both script versions (Bash and Python) plus the bulk directory-creation script.
- A dated terminal screenshot showing both versions run with at least two different threshold values, and the directory script run twice.
- A short written comparison (maximum half a page) of the two scripting approaches.

AI-restricted submission policy

All deliverables must include a dated screenshot (system clock visible) of your own terminal or IDE output, with your name and roll number visible in the prompt, filename, or a code comment. Where indicated, a short live demonstration or live search may be required in the lab session. Work that cannot be reproduced live by the student will not be marked.

Lab 3: Configuration Management with Ansible

Week	4
Mapped CLO	CLO-2 (Apply)
Tools / Environment	Ansible (open source); Docker (disposable target hosts); OpenSSH
Estimated Time	120 minutes

1. Objectives

On successful completion of this lab, the student will be able to:

- Explain agentless configuration management and how Ansible differs from scripting alone.
- Write and run an Ansible playbook against more than one target host.
- Use Ansible modules for package installation, service control, and templated files.
- Demonstrate idempotency from the play recap.

2. Preamble / Short Theory

Ansible (open source, GPL-3.0) is an agentless configuration-management tool: the control node connects over SSH and runs small Python modules on each target, so nothing needs to be pre-installed except SSH and Python. Playbooks describe the desired state declaratively in YAML, and modules check the current state before acting.

Docker containers serve as free, disposable target hosts. Authentication uses an SSH key pair instead of a password, which avoids hard-coded credentials (a Week 12 principle) and the need for `sshpass`.

3. Prerequisites

- Lab 2 completed.
- Docker installed (Docker Engine or Docker Desktop).
- Python 3 with the `venv` module.

4. Step-by-Step Guide

Step 1: Install Ansible in a virtual environment

```
mkdir lab03-ansible && cd lab03-ansible
python3 -m venv ~/.venvs/ansible
source ~/.venvs/ansible/bin/activate
pip install ansible
ansible --version
```

Step 2: Create an SSH key pair for the lab

```
ssh-keygen -t ed25519 -f id_ed25519_lab -N ""
printf 'id_ed25519_lab\n' > .gitignore
```

Step 3: Build the target image

Create a file named `Dockerfile` with the content below. The image contains an SSH server, Python 3, and a sudo-enabled user named `ansible` that accepts only your key.

```
FROM ubuntu:24.04

RUN apt-get update \
 && apt-get install -y --no-install-recommends openssh-server python3
sudo ca-certificates \
 && rm -rf /var/lib/apt/lists/* \
 && mkdir -p /run/sshd \
 && useradd -m -s /bin/bash ansible \
 && echo "ansible ALL=(ALL) NOPASSWD:ALL" > /etc/sudoers.d/ansible \
 && mkdir -p /home/ansible/.ssh && chmod 700 /home/ansible/.ssh

COPY id_ed25519_lab.pub /home/ansible/.ssh/authorized_keys
RUN chown -R ansible:ansible /home/ansible/.ssh \
 && chmod 600 /home/ansible/.ssh/authorized_keys

EXPOSE 22 80
CMD ["/usr/sbin/sshd", "-D"]
```

```
docker build -t ansible-target:1.0 .
```

Step 4: Start two target containers

Each container publishes SSH on its own host port and HTTP on its own host port.

```
docker run -d --name node1 -p 2221:22 -p 8181:80 ansible-target:1.0
docker run -d --name node2 -p 2222:22 -p 8182:80 ansible-target:1.0
```

Step 5: Create the inventory and configuration

Create `inventory.ini` and `ansible.cfg`.

```
[webservers]
node1 ansible_host=127.0.0.1 ansible_port=2221
node2 ansible_host=127.0.0.1 ansible_port=2222

[webservers:vars]
ansible_user=ansible
ansible_ssh_private_key_file=./id_ed25519_lab
ansible_ssh_common_args='-o StrictHostKeyChecking=no -o
UserKnownHostsFile=/dev/null'
```

```
[defaults]
inventory = inventory.ini
host_key_checking = False
retry_files_enabled = False
```

Step 6: Test connectivity

```
ansible webservers -m ping
```

Both nodes must answer with `"ping": "pong"`. If not, see the troubleshooting hints at the end of this lab.

Step 7: Write the playbook

Create `site.yml` that installs nginx and ensures it is running on both hosts.

```
---
- name: Configure web servers
  hosts: webservers
  become: true
  vars:
    student_id: "Your Name - Roll No"
  tasks:
    - name: Install nginx
      ansible.builtin.apt:
        name: nginx
        state: present
        update_cache: true
        cache_valid_time: 3600

    - name: Ensure nginx is running
      ansible.builtin.service:
        name: nginx
        state: started
        use: service   # containers have no systemd, so use the init-
script backend
```

Step 8: Run the playbook and verify idempotency

Run the playbook twice and compare the PLAY RECAP lines. Explain why the second run reports `changed=0` and why that matters for configuration management.

```
ansible-playbook site.yml
ansible-playbook site.yml
```

Step 9: Extend the playbook with a template

Create `templates/index.html.j2` and append the extra task to `site.yml` (at the same indentation as the other tasks). Re-run the playbook, then check both pages in the browser or with `curl`.

```
<!DOCTYPE html>
<html>
  <head><title>{{ inventory_hostname }}</title></head>
  <body>
    <h1>Configured by Ansible</h1>
    <p>Host: {{ inventory_hostname }} | Owner: {{ student_id }}</p>
  </body>
</html>
```

```
    - name: Deploy a custom index page
      ansible.builtin.template:
        src: templates/index.html.j2
        dest: /var/www/html/index.html
        mode: "0644"

curl http://127.0.0.1:8181
curl http://127.0.0.1:8182
```

Step 10: Clean up

```
docker rm -f node1 node2
```

Troubleshooting Hints

- `UNREACHABLE ... Permission denied (publickey)`: confirm that you built the image after generating the key, and that `ansible_ssh_private_key_file` points to the private key (not the `.pub` file).
- `Connection refused`: run `docker ps` to check the containers are up and the ports match the inventory.
- Windows: run all commands inside WSL2, and keep the lab folder inside the Linux file system so that key-file permissions work.

5. Required Outcomes / Tasks from the Student

- `Dockerfile`, `inventory.ini`, `ansible.cfg`, `site.yml`, and `templates/index.html.j2`.
- A dated terminal screenshot of two playbook runs showing the second run with `changed=0`.
- A screenshot of `curl` output from both nodes after the template task.

> **AI-restricted submission policy**
> All deliverables must include a dated screenshot (system clock visible) of your own terminal or IDE output, with your name and roll number visible in the prompt, filename, or a code comment. Where indicated, a short live demonstration or live search may be required in the lab session. Work that cannot be reproduced live by the student will not be marked.

Lab 4: Automating Deployments with Puppet (OpenVox)

Week	4
Mapped CLO	CLO-2 (Apply)
Tools / Environment	OpenVox/Puppet open-source agent; Docker (disposable host)
Estimated Time	90 minutes

1. Objectives

On successful completion of this lab, the student will be able to:

- Explain Puppet's declarative, manifest-based approach and contrast it with Ansible's agentless model.
- Write a Puppet manifest that declares desired package, file, and service state.
- Apply a manifest locally and verify convergence and drift correction.

2. Preamble / Short Theory

Puppet pioneered the declarative-manifest model: you describe resources and their relationships, and the agent converges the node to that state. Since 2025 the open-source distribution of Puppet is maintained by the community as **OpenVox**; the `puppet` command and the manifest language are unchanged, so everything in this lab applies to both.

In production, agents pull a catalog from a server on a schedule and continuously enforce it. In this lab you use `puppet apply`, which compiles and applies a local manifest, to concentrate on the resource model.

3. Prerequisites

- Lab 3 completed.
- Docker installed.

4. Step-by-Step Guide

Step 1: Start a clean Ubuntu container

```
docker run -it --name puppet-lab --hostname puppet-lab ubuntu:24.04
bash
```

Step 2: Install the open-source agent

Inside the container, install the agent from the community repository. If the package URL has changed, follow the current instructions at voxpupuli.org/openvox/install.

```
apt-get update && apt-get install -y wget ca-certificates
wget https://apt.voxpupuli.org/openvox8-release-ubuntu24.04.deb
dpkg -i openvox8-release-ubuntu24.04.deb
apt-get update && apt-get install -y openvox-agent
export PATH=/opt/puppetlabs/bin:$PATH
puppet --version
```

Step 3: Write a basic manifest

Create `site.pp` declaring a package, a file, and a service (write it with `cat > site.pp` and paste, or install `nano` first).

```
package { 'nginx':
  ensure => installed,
}

file { '/var/www/html/index.html':
  ensure  => file,
  content => "Managed by Puppet\n",
  require => Package['nginx'],
}

service { 'nginx':
  ensure   => running,
  provider => 'init',   # no systemd inside a container
  require  => Package['nginx'],
}
```

Step 4: Preview, then apply

First run a dry run, which reports what would change without changing anything, then apply.

```
puppet apply --noop site.pp
puppet apply site.pp
```

Step 5: Re-apply and observe convergence

Run `puppet apply site.pp` again and note that no resource changes are reported because the node has already converged.

```
puppet apply site.pp
```

Step 6: Correct drift

Change the managed file by hand and apply again. Puppet restores the declared content.

```
echo 'tampered' > /var/www/html/index.html
puppet apply site.pp
cat /var/www/html/index.html
```

Step 7: Compare with Lab 3

Write a short comparison of Puppet's resource model versus Ansible's playbook model, covering agent requirements, language, ordering (explicit `require` versus top-to-bottom tasks), and learning curve.

5. Required Outcomes / Tasks from the Student

- `site.pp`.
- A dated terminal screenshot of two `puppet apply` runs showing convergence (no changes on the second run) and the drift-correction run.
- A short written comparison (maximum half a page) of Puppet versus Ansible.

AI-restricted submission policy

All deliverables must include a dated screenshot (system clock visible) of your own terminal or IDE output, with your name and roll number visible in the prompt, filename, or a code comment. Where indicated, a short live demonstration or live search may be required in the lab session. Work that cannot be reproduced live by the student will not be marked.

Lab 5: Container Orchestration Basics

Week	5
Mapped CLO	CLO-2 (Apply)
Tools / Environment	Docker; Kubernetes via Minikube; kubectl
Estimated Time	90 minutes

1. Objectives

On successful completion of this lab, the student will be able to:

- Explain the role of container orchestration in automated infrastructure.
- Deploy a containerized application to a local Kubernetes cluster.
- Scale, update, roll back, and observe self-healing using kubectl.

2. Preamble / Short Theory

Kubernetes (open source, CNCF, Apache 2.0) automates deployment, scaling, and recovery of containerized applications by continuously reconciling actual state with the desired state you declare. Minikube runs a single-node cluster on your laptop, so no cloud account or billing is involved.

3. Prerequisites

- Lab 1 completed (general command-line comfort).
- Docker installed and running.
- Minikube and kubectl installed (both open source).

4. Step-by-Step Guide

Step 1: Start Minikube

```
minikube start --driver=docker
```

Step 2: Verify the cluster

```
kubectl get nodes
kubectl cluster-info
```

Step 3: Deploy a sample application

```
kubectl create deployment web --image=nginx:stable
kubectl expose deployment web --port=80 --type=NodePort
kubectl get deployments,pods,services
```

Step 4: Access the service

With the Docker driver on macOS and Windows, this command must be left running in its own terminal while you test.

```
minikube service web --url
```

Step 5: Scale the deployment

```
kubectl scale deployment web --replicas=3
kubectl get pods -o wide
```

Step 6: Observe self-healing

Delete one pod and watch the Deployment's controller replace it to restore three replicas. Replace the pod name with one from your own output.

```
kubectl delete pod <one-of-your-pod-names>
kubectl get pods -w
```

Step 7: Perform a rolling update

```
kubectl set image deployment/web nginx=nginx:1.27
kubectl rollout status deployment/web
```

Step 8: Roll back the update

```
kubectl rollout undo deployment/web
kubectl rollout history deployment/web
```

Step 9: Export the declarative form

Save the live Deployment as YAML. This is the same desired state expressed as code, which links this lab to the IaC principles of Week 2.

```
kubectl get deployment web -o yaml > web-deployment.yaml
```

Step 10: Clean up

```
kubectl delete service web
kubectl delete deployment web
minikube stop
```

5. Required Outcomes / Tasks from the Student

- The exact sequence of `kubectl` commands you used, saved as a shell script or text file, and `web-deployment.yaml`.
- A dated terminal screenshot showing three running pods after scaling, the pod replaced after deletion, and the rollout history after the rollback.

> **AI-restricted submission policy**
> All deliverables must include a dated screenshot (system clock visible) of your own terminal or IDE output, with your name and roll number visible in the prompt, filename, or a code comment. Where indicated, a short live demonstration or live search may be required in the lab session. Work that cannot be reproduced live by the student will not be marked.

Lab 6: Infrastructure Provisioning with Terraform / OpenTofu

Week	6
Mapped CLO	CLO-2 (Apply)
Tools / Environment	OpenTofu (or Terraform); Docker provider; Git
Estimated Time	120 minutes

1. Objectives

On successful completion of this lab, the student will be able to:

- Write a configuration that provisions multiple related resources.
- Use variables, outputs, and implicit dependencies.
- Apply, modify, and destroy infrastructure with the standard workflow, and read an incremental plan.

2. Preamble / Short Theory

This lab returns to the tool from Lab 1 and goes deeper into real provisioning. To keep it free and reproducible, the open-source Docker provider provisions a network, an image, and containers: the same workflow used to provision AWS, Azure, or GCP resources when credentials are available. Because the containers reference the network and the image, OpenTofu builds a dependency graph and orders creation and destruction automatically.

3. Prerequisites

- Labs 1 and 5 completed.
- Docker installed and running.

4. Step-by-Step Guide

Step 1: Set up the project

```
mkdir lab06-terraform-provisioning && cd lab06-terraform-provisioning
git init -b main
printf '.terraform/\n*.tfstate\n*.tfstate.*\ntfplan\n' > .gitignore
```

Step 2: Declare the provider (versions.tf)

```
terraform {
  required_version = ">= 1.6.0"
  required_providers {
    docker = {
      source  = "kreuzwerker/docker"
      version = "~> 3.0"
    }
  }
}

provider "docker" {}
```

Step 3: Add variables (variables.tf)

```
variable "container_count" {
  description = "Number of web containers to provision"
  type        = number
  default     = 2
}

variable "external_base_port" {
  description = "Host port of the first container; each next one adds
1"
  type        = number
  default     = 8080
}
```

Step 4: Provision a network, an image, and containers (main.tf)

```
resource "docker_network" "lab" {
  name = "lab06-net"
}

resource "docker_image" "nginx" {
  name         = "nginx:stable"
  keep_locally = true
}

resource "docker_container" "web" {
  count = var.container_count
  name  = "web-${count.index}"
  image = docker_image.nginx.image_id

  networks_advanced {
    name = docker_network.lab.name
  }

  ports {
    internal = 80
    external = var.external_base_port + count.index
  }
}
```

Step 5: Add outputs (outputs.tf) and a variable file

```
output "container_names" {
  value = docker_container.web[*].name
}

output "urls" {
  value = [for i in range(var.container_count) :
"http://localhost:${var.external_base_port + i}"]
}
```

Create `terraform.tfvars` containing:

```
container_count = 2
```

Step 6: Format, validate, and run the workflow

```
tofu fmt
tofu validate
tofu init
tofu plan -out=tfplan
tofu apply tfplan
curl http://localhost:8080
```

Step 7: Modify and re-apply

Change `container_count` to 4 in `terraform.tfvars`, then run `tofu plan -out=tfplan` and `tofu apply tfplan`. Observe that only two new containers are added and the existing ones are untouched.

```
tofu plan -out=tfplan
tofu apply tfplan
tofu output
```

Step 8: Commit your work

```
git add .
git commit -m "Lab 6: Docker provisioning with OpenTofu"
```

Step 9: Destroy

```
tofu destroy
```

5. Required Outcomes / Tasks from the Student

- `versions.tf`, `variables.tf`, `main.tf`, `outputs.tf`, and `terraform.tfvars`.
- A dated terminal screenshot of `tofu apply` and of the `tofu plan` output after the `container_count` change, showing only 2 additions.
- Two sentences explaining how OpenTofu knew to create the network before the containers.

> **AI-restricted submission policy**
> All deliverables must include a dated screenshot (system clock visible) of your own terminal or IDE output, with your name and roll number visible in the prompt, filename, or a code comment. Where indicated, a short live demonstration or live search may be required in the lab session. Work that cannot be reproduced live by the student will not be marked.

Lab 7: CI/CD Pipeline Automation

Week	11
Mapped CLO	CLO-2 (Apply)
Tools / Environment	Jenkins (open source, self-hosted); Git; Docker; GitHub Actions free tier (optional)
Estimated Time	150 minutes

1. Objectives

On successful completion of this lab, the student will be able to:

- Explain the stages of a CI/CD pipeline as applied to infrastructure automation.
- Run a self-hosted Jenkins server with OpenTofu available to its pipelines.
- Run an automated format-check, validate, plan, approve, and apply pipeline from a Jenkinsfile stored in Git.

2. Preamble / Short Theory

The course outline lists both Jenkins and GitHub Actions. This manual teaches Jenkins as the primary tool because it is open source (MIT license) and self-hosted, so every student can build real pipelines without a hosted-service account; the GitHub Actions free tier is shown as an optional alternative.

The standard Jenkins image contains neither OpenTofu nor access to Docker, so you first build a small custom image. The pipeline includes a manual approval gate before apply, the safeguard discussed in Week 11.

3. Prerequisites

- Lab 6 completed, with its Git repository on branch `main`.
- Docker installed. Port 8090 free on the host (Jenkins uses 8090 so that it does not clash with the web containers of Lab 6, which use ports 8080 and up).

4. Step-by-Step Guide

Step 1: Add the Jenkinsfile to your Lab 6 repository

Create a file named `Jenkinsfile` in the root of `lab06-terraform-provisioning` and commit it.

```
pipeline {
  agent any
  stages {
    stage('Init')          { steps { sh 'tofu init -input=false' } }
    stage('Format Check') { steps { sh 'tofu fmt -check -
recursive' } }
    stage('Validate')      { steps { sh 'tofu validate' } }
    stage('Plan')          { steps { sh 'tofu plan -input=false -
out=tfplan' } }
    stage('Approval') {
      steps { input message: 'Review the plan above. Apply it?', ok:
'Apply' }
    }
    stage('Apply')         { steps { sh 'tofu apply -input=false
tfplan' } }
  }
}
```

```
cd ~/lab06-terraform-provisioning
git add Jenkinsfile
git commit -m "Add Jenkins pipeline"
```

Step 2: Build a Jenkins image that contains OpenTofu

In a new folder `lab07-jenkins`, create a `Dockerfile`. Check opentofu.org for the current release and adjust `TOFU_VERSION` if you prefer a newer one.

```
FROM jenkins/jenkins:lts
USER root
ARG TOFU_VERSION=1.9.0
RUN apt-get update \
 && apt-get install -y --no-install-recommends unzip curl git \
 && ARCH=$(dpkg --print-architecture) \
 && curl -fsSL -o /tmp/tofu.zip
"https://github.com/opentofu/opentofu/releases/download/v${TOFU_VERSIO
N}/tofu_${TOFU_VERSION}_linux_${ARCH}.zip" \
 && unzip /tmp/tofu.zip tofu -d /usr/local/bin \
 && rm -f /tmp/tofu.zip \
 && rm -rf /var/lib/apt/lists/* \
 && git config --system --add safe.directory '*'
```

Step 3: Run Jenkins

The container runs as root for the lab so that it can use the Docker socket, and it mounts your Lab 6 repository read-only (adjust the path in the last `-v` option if your repository is somewhere else). Do not use this configuration in production.

```
docker build -t jenkins-iac:lab7 .
docker run -d --name jenkins -u root -p 8090:8080 -p 50000:50000 \
  -v jenkins_home:/var/jenkins_home \
  -v /var/run/docker.sock:/var/run/docker.sock \
  -v "$HOME/lab06-terraform-provisioning:/repos/lab06:ro" \
  jenkins-iac:lab7
```

Step 4: Complete the setup wizard

Retrieve the initial admin password, browse to http://127.0.0.1:8090, install the suggested plugins, and create an admin user.

```
docker exec jenkins cat /var/jenkins_home/secrets/initialAdminPassword
```

Step 5: Create a Pipeline job

New Item, Pipeline, name `lab07-iac-pipeline`. Under Pipeline choose Pipeline script from SCM, SCM Git, Repository URL `file:///repos/lab06`, Branch `*/main`, Script Path `Jenkinsfile`. Save.

Step 6: Trigger the pipeline

Click Build Now. Watch each stage run in the Stage View. At the Approval stage, open the build, review the plan in the console output, and click Apply.

Step 7: Review the console output

Open the Console Output of a completed build and identify exactly which `tofu` commands Jenkins executed and in what order. Verify the containers with `docker ps`. The pipeline keeps its own state inside the Jenkins workspace, so clean up from there:

```
docker exec -w /var/jenkins_home/workspace/lab07-iac-pipeline jenkins
tofu destroy -auto-approve
```

Step 8: (Optional) Mirror the pipeline in GitHub Actions

Create `.github/workflows/opentofu.yml` in a GitHub repository containing your Lab 6 code, push, and compare the setup effort with self-hosted Jenkins.

```
name: OpenTofu pipeline
on: [push]
jobs:
  iac:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: opentofu/setup-opentofu@v1
      - run: tofu init -input=false
      - run: tofu fmt -check -recursive
      - run: tofu validate
      - run: tofu plan -input=false
```

5. Required Outcomes / Tasks from the Student

- The `Jenkinsfile`, the `Dockerfile`, and the console output log of at least one full, successful pipeline run.

- A dated screenshot of the Jenkins Stage View showing all stages passing, including the approval gate.

AI-restricted submission policy
All deliverables must include a dated screenshot (system clock visible) of your own terminal or IDE output, with your name and roll number visible in the prompt, filename, or a code comment. Where indicated, a short live demonstration or live search may be required in the lab session. Work that cannot be reproduced live by the student will not be marked.

Lab 8: Automated Monitoring and Logging

Week	10
Mapped CLO	CLO-2 (Apply)
Tools / Environment	Prometheus; Grafana; node_exporter; Docker
Estimated Time	120 minutes

1. Objectives

On successful completion of this lab, the student will be able to:

- Explain the role of automated monitoring and logging in infrastructure reliability.
- Deploy Prometheus to scrape metrics from a running service.
- Build a Grafana dashboard and an alert rule that visualize and watch those metrics.
- Inspect container logs and explain why centralized logging matters.

2. Preamble / Short Theory

Prometheus (open source, CNCF) and Grafana (open source) are the de facto standard, freely self-hostable monitoring stack. Prometheus pulls (scrapes) metrics from HTTP endpoints; Grafana visualizes them and can raise alerts. All containers join one Docker network and find each other by name, which works identically on Linux, macOS, and Windows (the name `host.docker.internal` does not exist on Linux by default).

This lab concentrates on metrics. Log aggregation (for example with Grafana Loki) is discussed in the Week 10 lecture and can be explored as an extension.

3. Prerequisites

- Docker installed.
- Lab 5 or Lab 6 completed (a running containerized service is helpful but not required).

4. Step-by-Step Guide

Step 1: Create the Prometheus configuration

In a new folder `lab08-monitoring`, create `prometheus.yml`.

```
global:
  scrape_interval: 15s

scrape_configs:
  - job_name: 'prometheus'
    static_configs:
      - targets: ['localhost:9090']

  - job_name: 'node'
    static_configs:
      - targets: ['node-exporter:9100']
```

Step 2: Start the monitoring stack

```
docker network create monitoring
docker run -d --name node-exporter --network monitoring -p 9100:9100
prom/node-exporter
docker run -d --name prometheus --network monitoring -p 9090:9090 \
  -v "$(pwd)/prometheus.yml:/etc/prometheus/prometheus.yml:ro" \
  prom/prometheus
docker run -d --name grafana --network monitoring -p 3000:3000
grafana/grafana
```

Step 3: Verify the targets

Browse to http://127.0.0.1:9090/targets and confirm that both the `prometheus` and `node` targets show state UP.

Step 4: Query metrics

In the Prometheus expression browser, run the two queries below and note what each returns.

```
100 - (avg by (instance)
(rate(node_cpu_seconds_total{mode="idle"}[2m])) * 100)

node_memory_MemAvailable_bytes / node_memory_MemTotal_bytes * 100
```

Step 5: Add Prometheus as a Grafana data source

Open http://127.0.0.1:3000 (login `admin` / `admin`, then set a new password). Add a Prometheus data source with URL `http://prometheus:9090` and click Save and Test.

Step 6: Build a dashboard

Create a dashboard with two panels, CPU usage and memory available, using the queries from Step 4 (or `node_cpu_seconds_total` and `node_memory_MemAvailable_bytes` directly). Set the refresh to 15 seconds.

Step 7: Set an alert rule

Create a Grafana alert that fires when memory availability falls below a threshold you choose, with a pending period of 2 minutes. Document the threshold and your rationale.

Step 8: Inspect logs

```
docker logs --tail 20 prometheus
docker logs --since 5m grafana | head -n 20
```

Explain in two sentences why reading logs one container at a time would not scale to fifty containers.

Step 9: Export the dashboard

Use Dashboard settings, JSON Model, and save the JSON to `dashboard.json`.

5. Required Outcomes / Tasks from the Student

- `prometheus.yml` and `dashboard.json`.
- A dated screenshot of the Prometheus targets page showing UP status and of the live Grafana dashboard.
- Your chosen alert threshold with a one-paragraph justification.

AI-restricted submission policy

All deliverables must include a dated screenshot (system clock visible) of your own terminal or IDE output, with your name and roll number visible in the prompt, filename, or a code comment. Where indicated, a short live demonstration or live search may be required in the lab session. Work that cannot be reproduced live by the student will not be marked.

Lab 9: Security in Infrastructure Automation

Week	12
Mapped CLO	CLO-3 (Evaluate)
Tools / Environment	OpenSCAP; Lynis; Docker or a Linux VM
Estimated Time	120 minutes

1. Objectives

On successful completion of this lab, the student will be able to:

- Evaluate the security and compliance posture of a target using open-source scanning tools.
- Run and interpret an OpenSCAP compliance scan.
- Run and interpret a Lynis hardening audit.
- Propose automated remediation for the findings.

2. Preamble / Short Theory

Automation that provisions infrastructure quickly can just as quickly propagate a misconfiguration. OpenSCAP (open source, Red Hat) and Lynis (open source, CISOfy) are free compliance and hardening auditors that let you evaluate, not just apply, automation outcomes.

A container shares the host kernel and has no boot loader, so some rules are not applicable and will be reported as such. Treat the container as a teaching target and say so in your evaluation; a Vagrant VM gives more complete results if you have one.

3. Prerequisites

- Docker installed (or a Debian/Ubuntu VM you can scan as root).
- Lab 3 completed (for the remediation discussion).

4. Step-by-Step Guide

Step 1: Start a scan target and install the tools

```
docker run -it --name scan-target debian:12 bash

# inside the container:
apt-get update
apt-get install -y openscap-scanner ssg-debian lynis procps
cd /root
```

Step 2: Locate the compliance content and list profiles

The exact data-stream file name depends on the Debian release, so list the directory first and use the name you see. Choose one profile ID printed by `oscap info`.

```
ls /usr/share/xml/scap/ssg/content/
DS=/usr/share/xml/scap/ssg/content/ssg-debian12-ds.xml    # use the
file name printed above
oscap info "$DS"                                           # lists the
available profile IDs

oscap xccdf eval \
  --profile <PROFILE_ID_FROM_OSCAP_INFO> \
  --results results.xml --report report.html \
  "$DS"
# exit status 2 simply means "some rules failed" - that is expected
```

Step 3: Review the HTML report

Copy `report.html` out of the container and open it in a browser. Identify the three highest-severity failed rules and, for each, note what the rule checks and why it matters.

```
docker cp scan-target:/root/report.html ./report.html
docker cp scan-target:/var/log/lynis-report.dat ./lynis-report.dat
```

Step 4: Run Lynis

Back inside the container, run the audit and extract the hardening index and the top five suggestions.

```
lynis audit system
grep hardening_index /var/log/lynis-report.dat
grep '^suggestion\[\]' /var/log/lynis-report.dat | head -n 5
```

Step 5: Write an evaluation

In a short report (maximum one page), evaluate whether the scanned target is fit for production use, citing specific findings from both tools, and propose which automation step (an Ansible task, a Puppet resource, or a Terraform setting) could remediate each issue.

5. Required Outcomes / Tasks from the Student

- `report.html` (OpenSCAP) and the Lynis report or log.
- The written evaluation (maximum one page) with specific, cited findings and proposed automated remediation.
- A dated screenshot of both scans running.

> **AI-restricted submission policy**
> All deliverables must include a dated screenshot (system clock visible) of your own terminal or IDE output, with your name and roll number visible in the prompt, filename, or a code comment. Where indicated, a short live demonstration or live search may be required in the lab session. Work that cannot be reproduced live by the student will not be marked.

Lab 10: Multi-Cloud Automation Case Study

Week	13
Mapped CLO	CLO-3 (Evaluate)
Tools / Environment	OpenTofu; Ansible; Docker provider
Estimated Time	150 minutes

1. Objectives

On successful completion of this lab, the student will be able to:

- Design a provisioning-plus-configuration workflow that could target more than one environment or provider.
- Structure an OpenTofu project with a reusable module and two environments.
- Connect provisioning to configuration by generating an Ansible inventory from OpenTofu outputs.
- Evaluate trade-offs between modular, provider-agnostic design and duplicated per-environment code.

2. Preamble / Short Theory

Enterprises rarely commit to one cloud. This lab does not require paid accounts: it uses the Docker provider to simulate two environments (staging and production) from one reusable module, so the design and evaluation skills transfer directly to real AWS, Azure, or GCP setups.

Provisioning (OpenTofu) and configuration (Ansible) stay separate. OpenTofu writes the inventory file, so Ansible always sees the resources that actually exist.

3. Prerequisites

- Labs 3 and 6 completed.
- Ansible installed in your virtual environment (the `ansible` package includes the Docker connection plugin), and the Docker CLI available.

4. Step-by-Step Guide

Step 1: Design the module structure

Sketch (on paper or in a diagram tool) the layout below: one module, two environment folders that call it with different variables, and a shared inventory template.

```
lab10-multienv/
  modules/web/{versions.tf,variables.tf,main.tf,outputs.tf}
  staging/main.tf
  production/main.tf
  templates/inventory.tftpl
  site.yml
```

Step 2: Create the reusable module

Create the four files in `modules/web/`.

```
terraform {
  required_providers {
    docker = {
      source  = "kreuzwerker/docker"
      version = "~> 3.0"
    }
  }
}

variable "container_count" {
  type = number
}

variable "environment" {
  type = string
}

variable "image" {
  type    = string
  default = "ubuntu:24.04"
}

resource "docker_image" "base" {
  name         = var.image
  keep_locally = true
}

resource "docker_container" "web" {
  count    = var.container_count
  name     = "${var.environment}-web-${count.index}"
  image    = docker_image.base.image_id
  command  = ["sleep", "infinity"]
  must_run = true
}

output "container_names" {
  value = docker_container.web[*].name
}
```

Step 3: Create the inventory template

Create `templates/inventory.tftpl`.

```
[web]
%{ for h in hosts ~}
${h} ansible_connection=community.docker.docker
ansible_python_interpreter=/usr/bin/python3
%{ endfor ~}
```

Step 4: Call the module from two environments

Create `staging/main.tf` (module call with `container_count = 1` and `environment = "staging"`) and `production/main.tf` (`container_count = 3`, `environment =`

`"production"`). Each environment also declares the providers and writes its own inventory, as in the staging example below; change the two values for production.

```
terraform {
  required_providers {
    docker = {
      source  = "kreuzwerker/docker"
      version = "~> 3.0"
    }
    local = {
      source  = "hashicorp/local"
      version = "~> 2.4"
    }
  }
}

provider "docker" {}

module "web" {
  source          = "../modules/web"
  container_count = 1
  environment     = "staging"
}

resource "local_file" "inventory" {
  filename = "${path.module}/inventory.ini"
  content  =
templatefile("${path.module}/../templates/inventory.tftpl", {
    hosts = module.web.container_names
  })
}

output "container_names" {
  value = module.web.container_names
}
```

Step 5: Apply both environments independently

```
cd staging && tofu init && tofu apply && cd ..
cd production && tofu init && tofu apply && cd ..
```

Step 6: Add a post-provisioning Ansible step

Write `site.yml` that configures the containers identically in both environments, and run it against each generated inventory.

```
---
- name: Bootstrap Python on bare containers
  hosts: web
  gather_facts: false
  tasks:
    - name: Install python3 if it is missing
```

```
      ansible.builtin.raw: test -x /usr/bin/python3 || (apt-get update
&& apt-get install -y python3)
      register: bootstrap
      changed_when: "'Setting up' in bootstrap.stdout"

- name: Configure web servers identically in every environment
  hosts: web
  tasks:
    - name: Install nginx
      ansible.builtin.apt:
        name: nginx
        state: present
        update_cache: true
        cache_valid_time: 3600

    - name: Publish a page that names the host
      ansible.builtin.copy:
        content: "Hello from {{ inventory_hostname }}\n"
        dest: /var/www/html/index.html
        mode: "0644"

    - name: Ensure nginx is running
      ansible.builtin.service:
        name: nginx
        state: started
        use: service
```

```
ansible-playbook -i staging/inventory.ini site.yml
ansible-playbook -i production/inventory.ini site.yml
```

Step 7: Evaluate the design

Write a short evaluation (maximum one page) comparing this modular approach against writing separate, duplicated configurations per environment or provider. Discuss maintainability, drift risk, and the effort of onboarding a third environment.

5. Required Outcomes / Tasks from the Student

- The full module plus two-environment project structure and the Ansible playbook.
- A dated screenshot showing both environments applied with the correct differing container counts (1 and 3).
- The written evaluation (maximum one page).

> **AI-restricted submission policy**
> All deliverables must include a dated screenshot (system clock visible) of your own terminal or IDE output, with your name and roll number visible in the prompt, filename, or a code comment. Where indicated, a short live demonstration or live search may be required in the lab session. Work that cannot be reproduced live by the student will not be marked.

Lab 11: Disaster Recovery Automation

Week	13
Mapped CLO	CLO-3 (Evaluate)
Tools / Environment	Velero; MinIO; Restic; Minikube; kubectl
Estimated Time	150 minutes

1. Objectives

On successful completion of this lab, the student will be able to:

- Explain the role of automated backup and restore in infrastructure reliability.
- Configure Velero to back up and restore Kubernetes resources to an S3-compatible store.
- Back up and restore files with Restic and compare the two tools.
- Evaluate recovery time and completeness after a simulated disaster.

2. Preamble / Short Theory

Velero (open source, CNCF) backs up Kubernetes resources and, optionally, volume contents. For volume contents its file-system backup uses Kopia; the older Restic path was deprecated and removed from Velero's backup path in recent releases, so this lab uses Restic separately and as a standalone tool to show the underlying concept of encrypted, deduplicated snapshots.

A local MinIO server stands in for the S3 object store that Velero needs, so no cloud account is required. Because Velero runs inside the Minikube cluster, it reaches MinIO on your computer through the special name `host.minikube.internal`, not `127.0.0.1` (which would point back at the Velero pod itself).

3. Prerequisites

- Lab 5 completed (Minikube cluster available).
- Docker installed (to run MinIO).
- Restic installed (`sudo apt install restic` or `brew install restic`).

4. Step-by-Step Guide

Step 1: Start MinIO and create the bucket

```
docker run -d --name minio -p 9000:9000 -p 9001:9001 \
  -e "MINIO_ROOT_USER=minio" -e "MINIO_ROOT_PASSWORD=minio12345" \
  minio/minio server /data --console-address ":9001"
```

Open http://127.0.0.1:9001, log in with `minio` / `minio12345`, and create a bucket named `velero-backups`.

Step 2: Start the cluster and install the Velero CLI

Run `minikube start --driver=docker`. Download the latest release for your operating system from github.com/vmware-tanzu/velero/releases and add it to your PATH. Then check the compatibility table in the `velero-plugin-for-aws` README to find the plugin tag that matches your CLI version.

Step 3: Create the credentials file

Save as `credentials-velero` (a lab-only secret; never commit it).

```
[default]
aws_access_key_id = minio
aws_secret_access_key = minio12345
```

Step 4: Install Velero into the cluster

```
export AWS_PLUGIN_VERSION=vX.Y.Z   # pick the tag listed as compatible
with your Velero CLI

velero install \
  --provider aws \
  --plugins velero/velero-plugin-for-aws:$AWS_PLUGIN_VERSION \
  --bucket velero-backups \
  --secret-file ./credentials-velero \
  --backup-location-config
region=minio,s3ForcePathStyle="true",s3Url=http://host.minikube.intern
al:9000 \
  --use-volume-snapshots=false \
  --use-node-agent \
  --default-volumes-to-fs-backup

velero backup-location get     # the PHASE column must read Available
```

Step 5: Deploy something to back up

```
kubectl create deployment demo --image=nginx:stable
kubectl expose deployment demo --port=80
kubectl create configmap demo-config --from-literal=owner="Your Name -
Roll No"
kubectl get deployments,services,configmaps
```

Step 6: Create a backup

```
velero backup create demo-backup --include-namespaces default
velero backup describe demo-backup
velero backup get
```

Step 7: Simulate a disaster

```
kubectl delete deployment demo
kubectl delete service demo
kubectl delete configmap demo-config
kubectl get deployments,services,configmaps
```

Step 8: Restore and time the recovery

```
time velero restore create demo-restore --from-backup demo-backup --
wait
kubectl get deployments,services,configmaps
velero restore describe demo-restore
```

Step 9: Practice with Restic

Repeat the idea at file level: back up a folder, destroy it, and restore it from the encrypted repository. Note that Restic restores the original full path beneath the target folder.

```
mkdir -p ~/lab11-data && echo "important record" > ~/lab11-
data/file.txt
export RESTIC_PASSWORD='lab-only-password'      # lab use only; never
commit a real one
restic init -r ~/lab11-restic-repo
restic -r ~/lab11-restic-repo backup ~/lab11-data
rm -rf ~/lab11-data                              # simulated disaster
restic -r ~/lab11-restic-repo snapshots
restic -r ~/lab11-restic-repo restore latest --target ~/lab11-restored
find ~/lab11-restored -name file.txt -exec cat {} \;
```

Step 10: Evaluate recovery

Record how long the restore took and verify that every resource returned. Write a short evaluation (maximum one page) of what this strategy would and would not protect against (for example, corrupted data replicated into the backup, loss of the backup store itself, or a region-wide outage) and what additional automation would mitigate those gaps.

5. Required Outcomes / Tasks from the Student

- The exact `velero`, `kubectl`, and `restic` command sequence used.
- A dated screenshot showing the backup, the simulated deletion, and the successful restore.
- The written evaluation (maximum one page) on recovery completeness and remaining risk.

> **AI-restricted submission policy**
> All deliverables must include a dated screenshot (system clock visible) of your own terminal or IDE output, with your name and roll number visible in the prompt, filename, or a code comment. Where indicated, a short live demonstration or live search may be required in the lab session. Work that cannot be reproduced live by the student will not be marked.

Lab 12: Capstone Project: Design and Implement an IaC Solution

Week	15
Mapped CLO	CLO-3 (Evaluate)
Tools / Environment	OpenTofu; Ansible; Jenkins; any tools from Labs 1-11
Estimated Time	Teams of 2-3, multi-session

1. Objectives

On successful completion of this lab, the student will be able to:

- Design a complete, end-to-end Infrastructure as Code solution that integrates provisioning, configuration, CI/CD, monitoring, and security scanning.
- Implement it using only the free, open-source tooling from this manual.
- Evaluate and defend the design's scalability, reliability, and security in a live demonstration.

2. Preamble / Short Theory

The capstone combines every tool from the manual into one small but complete automated solution that runs on a single laptop. It mirrors real practice: a generated inventory connects provisioning to configuration, a pipeline runs the whole chain on each commit, and monitoring and scanning provide evidence that the result works and is reasonably safe.

3. Prerequisites

- All previous labs completed and their tools available.
- A Git repository for the project.
- Teams of 2-3 students, as agreed with the instructor.

4. Step-by-Step Guide

Step 1: Define the scenario

In your team, choose a small but complete scenario, for example: "provision and configure a three-tier web application (load balancer, application containers, monitoring) that is reproducible from a single command." Get it approved before building.

Step 2: Design the architecture

Produce a one-page architecture diagram showing every component, which tool provisions or configures it, and the data and control flow between them (OpenTofu to Ansible to Jenkins to Prometheus/Grafana to the security scan).

Step 3: Implement provisioning

Write the OpenTofu configuration, reusing the module patterns of Labs 6 and 10. Validate inputs, pin provider versions, and expose outputs.

Step 4: Implement configuration management

Write Ansible playbooks (or Puppet manifests) that configure the provisioned resources. Generate the inventory from OpenTofu rather than writing it by hand.

Step 5: Wire in CI/CD

Create a Jenkinsfile (Lab 7) that runs init, lint, security scan, plan, approval, apply, and configure on each commit. Your Jenkins image will need Ansible and the Docker CLI in addition to OpenTofu.

Step 6: Add monitoring and security evaluation

Instrument the solution with Prometheus (and Grafana if time allows), and scan the code with a static analyzer such as Checkov and, where applicable, the running targets with Lynis (Lab 9).

Step 7: Run an end-to-end test

Trigger the full pipeline from a fresh commit and confirm that the infrastructure is provisioned, configured, monitored, and passes your baseline scan. Run the pipeline a second time and confirm it changes nothing.

Step 8: Prepare the demonstration

Prepare a 10-15 minute live demonstration: trigger the pipeline live, show the running infrastructure, and walk through the monitoring and scan results while defending your design decisions.

Assessment Criteria (CLO-3)

Criterion	What the instructor looks for
Correctness and function	The pipeline provisions, configures, and verifies the infrastructure end to end, and a second run changes nothing.
Design quality	Reusable modules, validated inputs, clear provisioning-versus-configuration boundary, generated (not hand-written) inventory.
Best practice and security	At least one Week 14 practice (linting, version pinning, documentation) and no hard-coded secrets (Week 12), supported by a scan.
Evaluation and justification	A written, evidence-based discussion of scalability, reliability, and security, and of the trade-offs you chose.
Live demonstration	Every member can explain and modify any part of the solution on request.

5. Required Outcomes / Tasks from the Student

- The project repository link, the architecture diagram, and a project report (maximum 6 pages) covering design decisions, trade-offs, and an evaluation of scalability, reliability, and security.
- A live, in-person demonstration of the working pipeline; no pre-recorded video will be accepted.
- Each team member must be able to explain and modify any part of the solution live if asked.

AI-restricted submission policy

All deliverables must include a dated screenshot (system clock visible) of your own terminal or IDE output, with your name and roll number visible in the prompt, filename, or a code comment. Where indicated, a short live demonstration or live search may be required in the lab session. Work that cannot be reproduced live by the student will not be marked.